Doker 101



<- fajne tło dla “nowoczenego stylu”

Czym jest?

W uproszczeniu: tworzenie mini - systemów operacyjnych, gdzie jedna apka, to jeden kontener, niezależnie od siebie, chyba że chcemy inaczej.

This is docker!

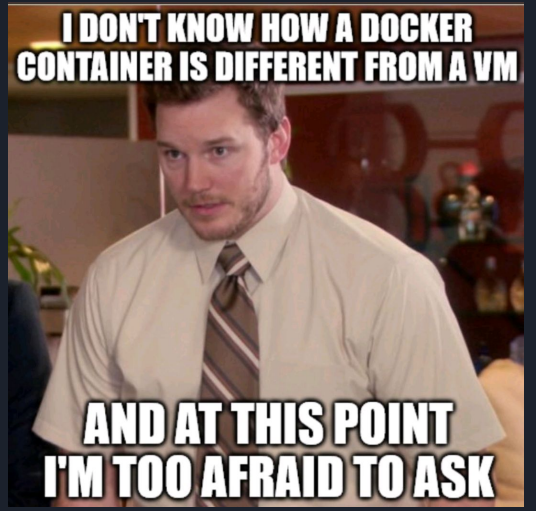
Docker emphasises on **isolation** of applications inside containers, so that different applications have no effect on each other.

Docker is smart.

Be like docker.

docker

#staysafe #fightcorona





Po co?

Ułatwienie budowania aplikacji, zminimalizowanie problemów z niekompatybilnością, łatwe pobieranie i używanie publicznych obrazów np. MySQL.

Maszyna Wirtualna (VM) vs Kontener – o co chodzi?

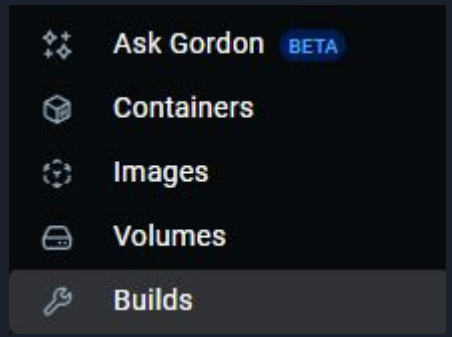


VM (Virtual Machine): Emuluje cały sprzęt. Potrzebuje osobnego, ciężkiego systemu operacyjnego dla każdej aplikacji. Zżera dużo RAMu i długo się odpala.

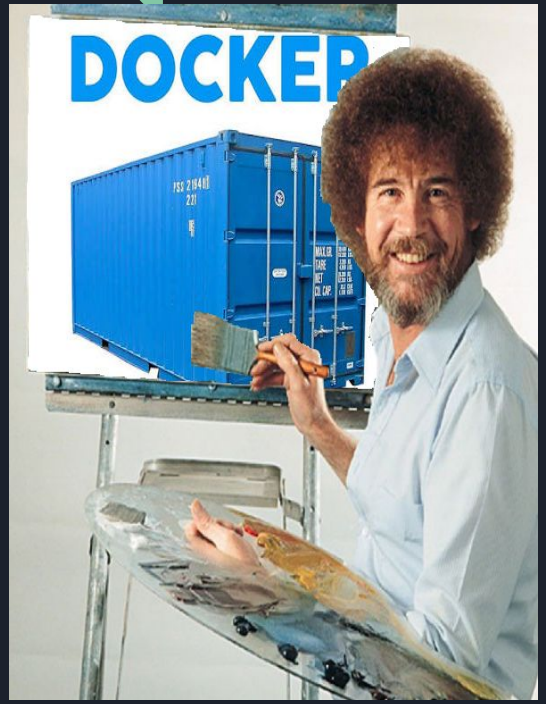
Kontener (Docker): Nie emuluje sprzętu. Współdzieli jądro systemu (kernel) z komputerem. Jest superlekki, startuje w sekundy i zużywa tylko tyle zasobów, ile realnie potrzebuje apka.

Kilka słów co czym jest (na podstawie Docker Desktop)

- Ask Gordon - idk, jakieś AI od dockera, chyba
- Containers (kontenery) - to co odpalamy, pojedynczy lub zbiór mini systemów
- Images (obrazy) - to z czego odpalamy, podobnie jak np. instalka windowsa, zawiera rzeczy potrzebne do uruchomienia kontenera, typu paczki, komendy, itd.
- Volumes (woluminy) - bazy danych, tak w skrócie
- Builds (budowle?) - lista wybudowanych kontenerów, z informacji o budowlach, np. logi



Obrazy i jak je stworzyć?



1. stworzenie obrazu:
    a. publiczny:
        - “docker pull [NAZWA]” - pobieranie gotowca
    b. własny:
        - Dockerfile - własny plik definiujący tworzenie obrazu
        - “docker build -t [super_nazwa] .” - komenda budowania obrazu
2. wystartowanie obrazu
    - docker run [super_nazwa]

Dockerfile i jak go stworzyć?

Dockerfile odpowiada za środowisko pojedynczego obrazu. Składowe:

“FROM node:18-alpine” – baza (od jakiegoś gotowego systemu/środowiska zaczynamy)

“WORKDIR /app” – folder roboczy wewnątrz kontenera

“COPY . .” – kopiuje pliki z komputera do kontenera

“RUN npm install” – odpala komendę podczas budowania

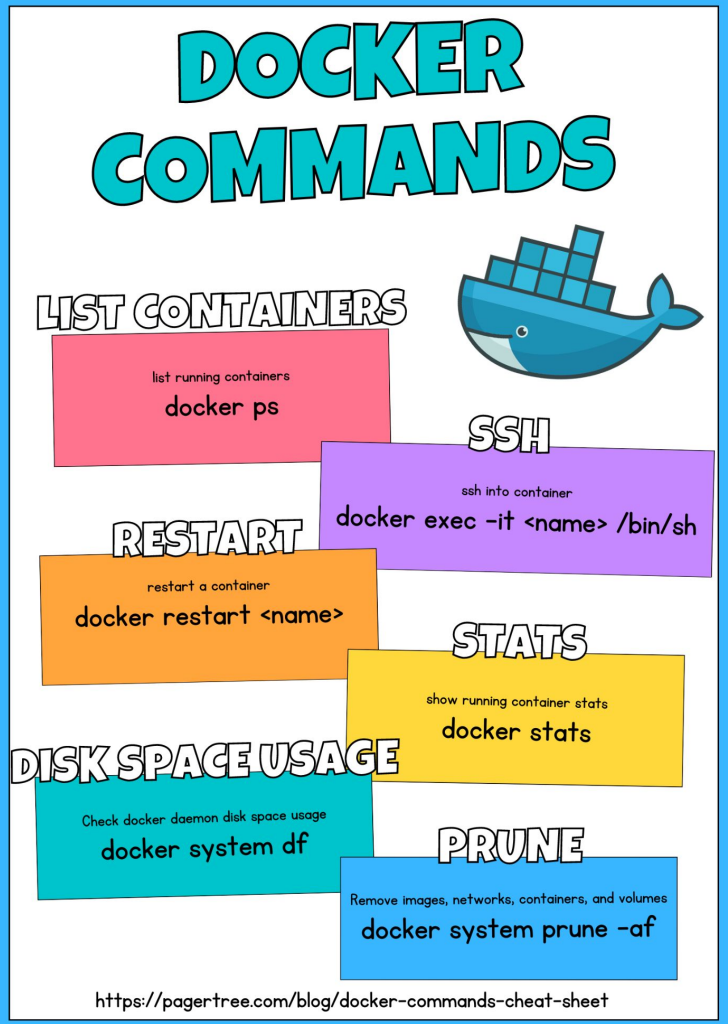
“CMD ["npm", "start"]” – komenda, która odpali się, z kontenerem

Dockerfile U

Dockerfile > ...

```
1  FROM node:14-alpine3.16
2
3  WORKDIR /app
4
5  COPY . .
6
7  RUN npm install
8
9  CMD [ "npm", "start" ]
```





Jakie komendy?

docker ps – wyświetla aktualnie działające kontenery

docker stop [ID] – grzeczne zatrzymanie kontenera

docker rm [ID] – usunięcie kontenera

docker rmi [NAZWA] – usunięcie obrazu z dysku

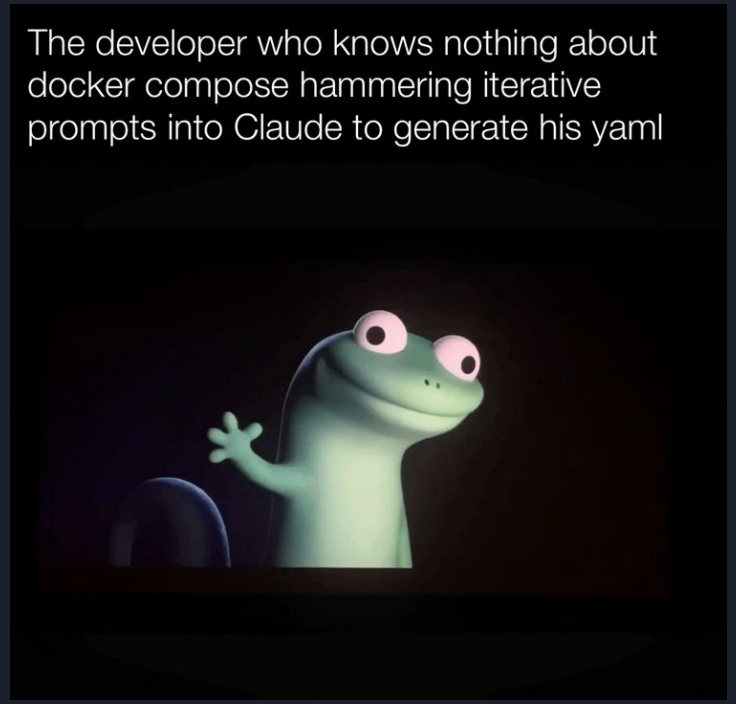
docker logs [ID] – pokazuje logi/błędy z działającego kontenera

Ale czy trzeba wszystko ręcznie i pojedynczo?

Problem: chcemy mieć frontend, backend, bazę danych i frytki do tego. Odpalenie tego pojedynczo i co gorsza ręcznie jest nieprzyjemne.

Rozwiązanie: inny ból głowy - docker-compose.yml

“docker-compose up” – stawia całe środowisko na raz.



docker-compose.yml i jak go zepsuć?

```yaml
version: '3.3'

services:
   db:
     image: mysql:5.7
     volumes:
       - db_data:C:\Users\User\Desktop\dcompose
     restart: always
     environment:
       MYSQL_ROOT_PASSWORD: rootwordpress
       MYSQL_DATABASE: wordpress
       MYSQL_USER: wordpress
       MYSQL_PASSWORD: wordpress

   wordpress:
     depends_on:
       - db
     image: wordpress:latest
     ports:
       - "8000:80"
     restart: always
     environment:
       WORDPRESS_DB_HOST: db:3306
       WORDPRESS_DB_USER: wordpress
       WORDPRESS_DB_PASSWORD: wordpress
volumes:
    db_data:
```

“services:” - lista kontenerów. Tutaj zgłaszasz, kto w ogóle bierze udział w projekcie

“db: / wordpress” / cokolwiek:” – nazwy kontenerów. Kontenery mogą się porumiewać między sobą tymi nazwami, zamiast URL.

“image:” – skąd bierzemy obraz.

“ports:” – zapisywane jako [komp]:[kontener], ustawia port dla kontenera

“environment:” – zmienne środowiskowe wewnątrz kontenera (taki .env)

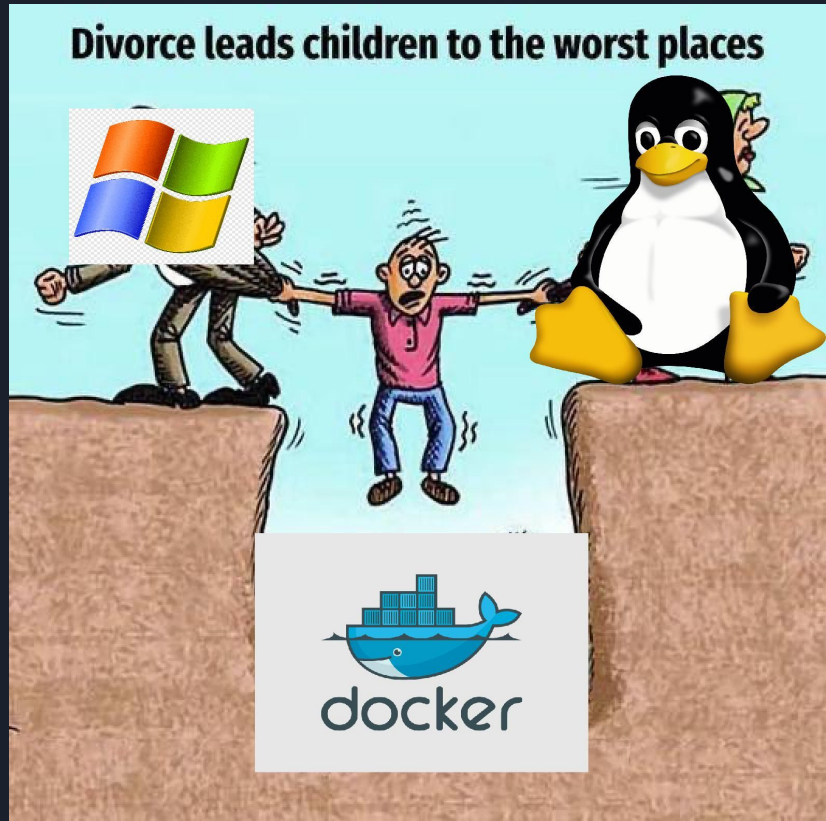
“depends_on:” – ustawianie kolejki (co musi się odpalić, by teraz ten się odpalił)

“volumes:” - miejsce na dysku gdzie będą zapisywane dane z kontenera (by np. nie utracić danych z bazy)

Jak nie zepsuć Dockera?

Spoiler - on sam się psuje. Ale można też spróbować tego:

- lekkie obrazy - zamiast pełnego Ubuntu, wersje "Alpine"
- .dockerignore - działa jak .gitignore. Może wywalić śmieci jak .venv, .vstudio, .idea
- jedna rzecz = jeden kontener - lepiej nie robić pełnego stacka w jednym kontenerze, to idealna droga by coś się zepsuło



Pytania?

Autor: Tymoteusz Huszcza

Źródła:

- Google
- Google Images
- Gemini
- Dokumentacja Dockera

